

Knowledge Based Engineering (KBE)

AE4204

Dr.ir. G. La Rocca

Flight Performance & Propulsion

Group (FPP)

Lecture 3

- *Frame Based Systems (Part 3)*
- *Inference mechanism and methods*
- *From KB Systems to KBE systems*



Contents

- A definition of Knowledge Based Engineering (KBE)
- Knowledge Based Systems (KBS)
 - Working principles: *production rules* and *inference mechanism*
- Frame Based Systems (FBS)
 - The Object-Oriented paradigm applied to KBS
 - Inference mechanisms & *methods*
- Recap: Rule based vs Frame based system
- Engineering design needs
- From KB Systems to KBE

Recap from previous lectures

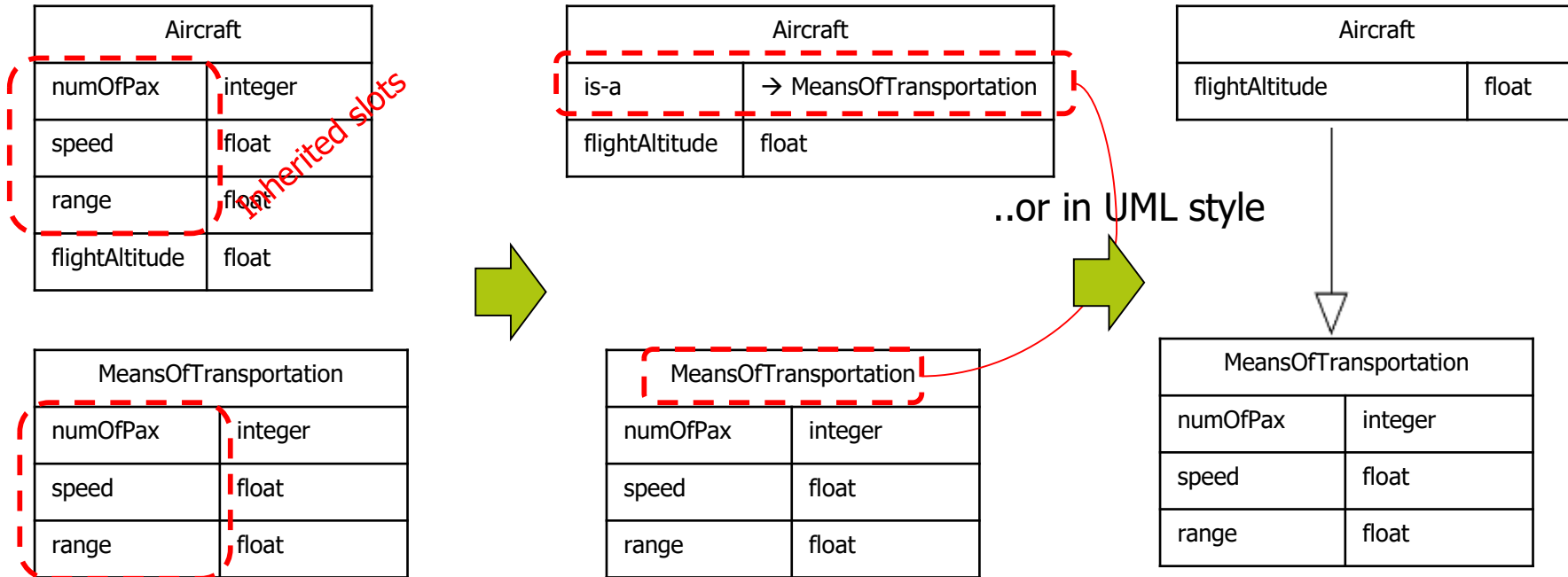
- Frame Based Systems offer a more advanced domain knowledge representation than rule-based systems, featuring concepts from the **Objected-Oriented modeling paradigm**
- The concepts of **class-frame** allows defining *few* conceptual templates which can be reused (*instantiated*) to model *many* individual **instance-frames**
- Class-frames can be connected* to other frames by means of **inheritance** and **composition/aggregation** links, thereby allowing complex-yet-efficient domain knowledge representations
- The UML (**Unified Modeling Language**), an industry-standard visual language specifically developed to support OO modeling, offers very effective diagrammatic representations of systems, their components and relationships



Frame Based Systems

The object oriented paradigm applied to knowledge based systems.

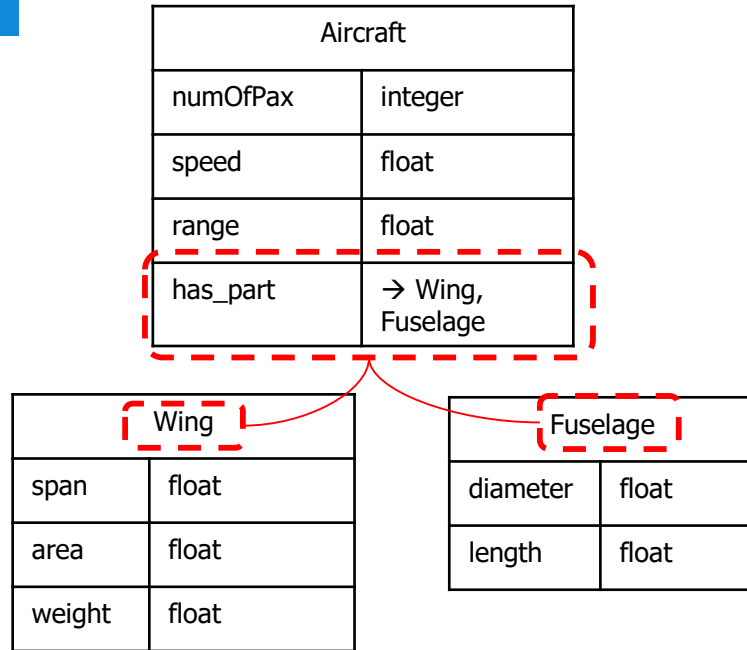
class-frames connected by **specialization/inheritance** links



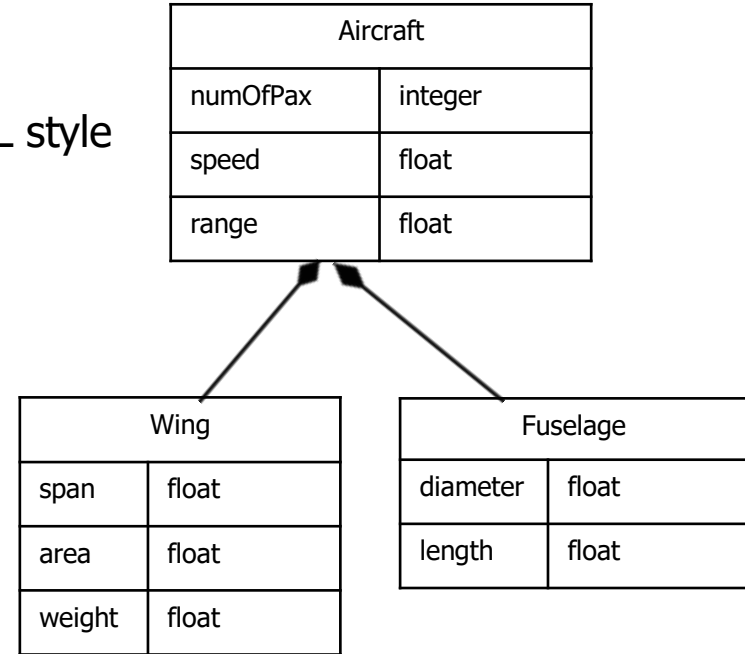
Frame Based Systems

The object oriented paradigm applied to knowledge based systems.

class-frames connected by **composition/aggregation** links



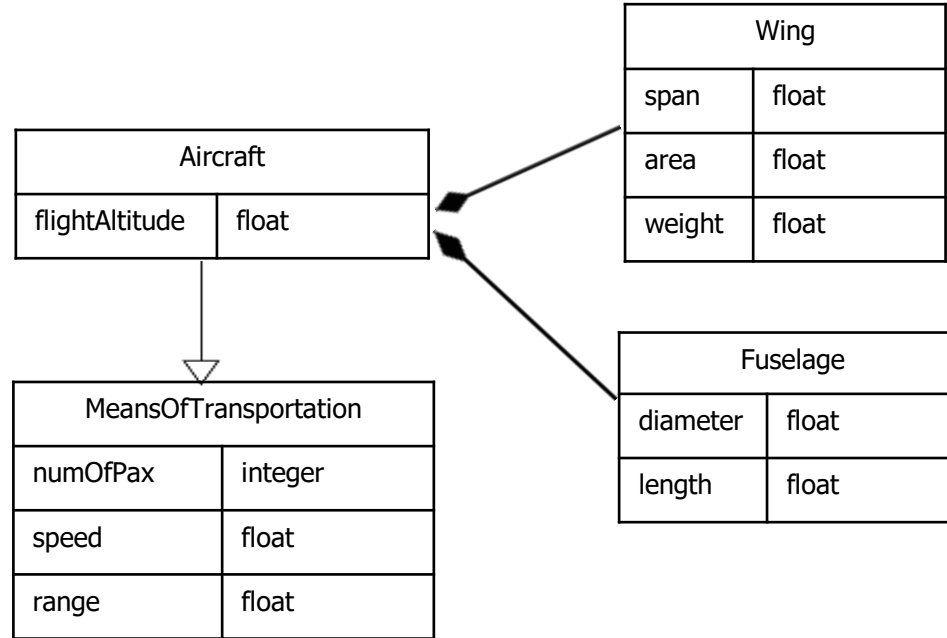
..or in UML style



Frame Based Systems

The object-oriented paradigm applied to knowledge based systems.

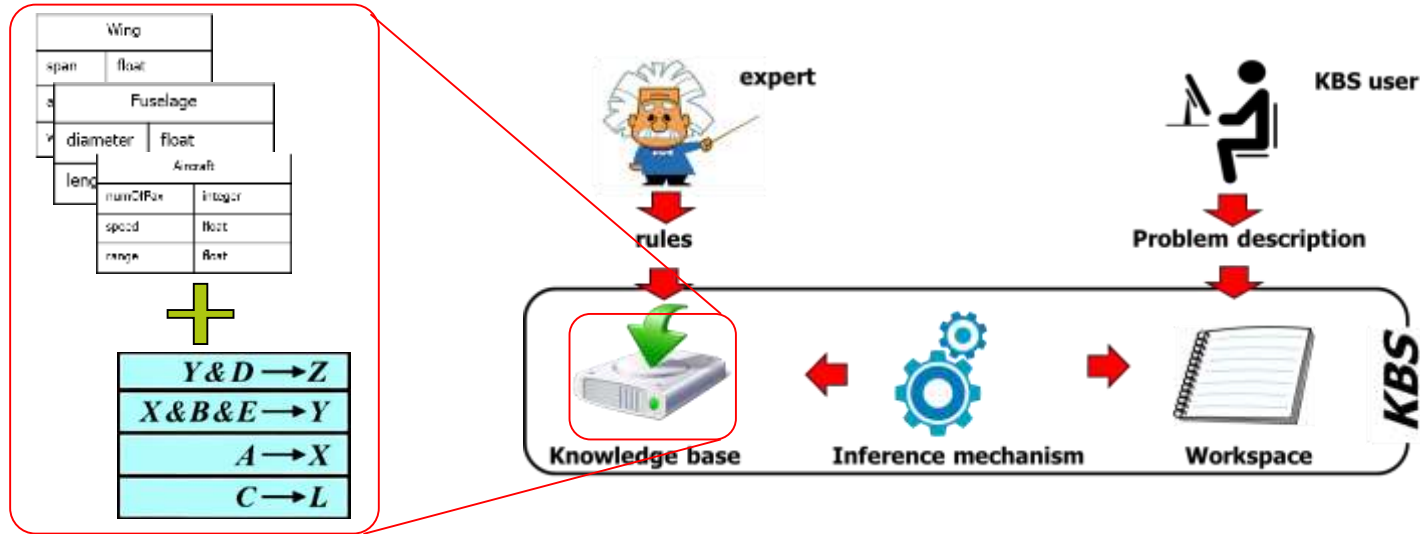
class-frames connected by **inheritance** and **composition** links



Frame Based Systems.

The object-oriented paradigm applied to knowledge based systems.

What about the mechanisms to manipulate and reason on knowledge in frame based systems?

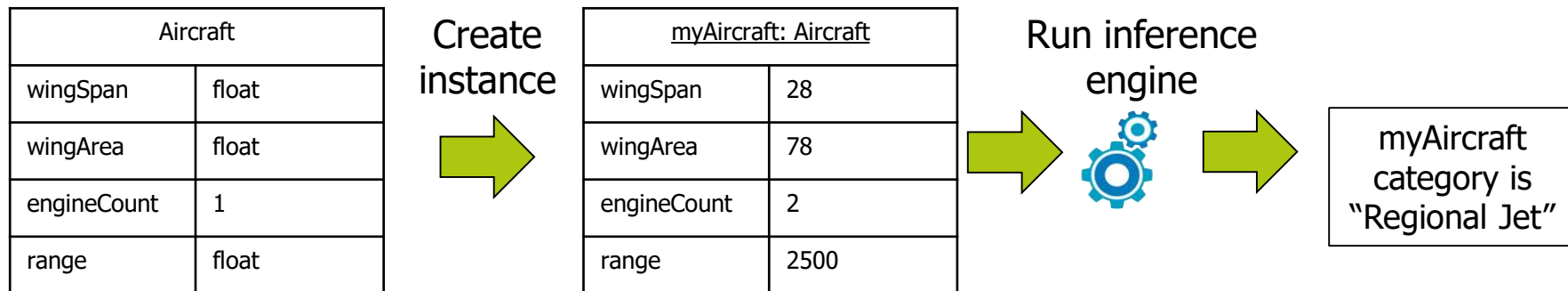




Inference mechanism in FBSs

The object-oriented paradigm applied to knowledge based systems.

Example of inference



IF engineCount = 2 AND range <= 3000km THEN category = "Regional Jet"

IF engineCount >=2 AND range > 8000km THEN category = "Long range Jet"

What do we observe? Rules operate on frames but are **stored outside frames**.
Same separation of knowledge and inference, as in pure rule-based systems. **However...**



Inference mechanism in FBSs

The object-oriented paradigm applied to knowledge based systems.

Apart from an explicit value, or a pointer to another frame, a frame slot can contain a **procedure**, called ***method****, that specifies how to evaluate the given slot value.

For example, through:

- the evaluation of an expression
- a series of actions to execute

Methods can include (simple) **data processing instructions** (e.g. mathematical operations) including **references to own or other frames slots**



Inference mechanism in FBSs

The object-oriented paradigm applied to knowledge based systems.

A frame slot can contain a **procedure**, that specifies how to compute the slot value, using slot values from the same frame or other frames

Aircraft	
numOfPax	integer
wingSpan	float
wingArea	float
aspectRatio	$\text{self.wingSpan}^2 / \text{self.wingArea}$

Aircraft	
numOfPax	integer
wingSpan	self.wing span
wingArea	self.wing area
aspectRatio	$\text{self.wingSpan}^2 / \text{self.wingArea}$
has_part	$\rightarrow$ Wing

Wing	
span	float
area	float
weight	float

note the **dot** notation to define the **reference chain**



Inference mechanism in FBSs

The object-oriented paradigm applied to knowledge based systems.

A frame slot can contain a **procedure**, that specifies how to compute the slot value, using slot values from the same frame or other frames

Aircraft	
numOfPax	integer
wingSpan	float
wingArea	float
aspectRatio	$\text{self.wingSpan}^2 / \text{self.wingArea}$

Aircraft	
numOfPax	integer
aspectRatio	$\text{self.wing.span}^2 / \text{self.wing.area}$
has_part	→ Wing

Wing	
span	float
area	float
weight	float

note the **dot** notation to define the **reference chain**



Inference mechanism in FBSs

The object-oriented paradigm applied to knowledge based systems.

A frame slot can contain a **procedure**, that specifies how to compute the slot value, using slot values from the same frame or other frames

Aircraft	
numOfPax	integer
wingSpan	float
wingArea	float
aspectRatio	$\text{self.wingSpan}^2 / \text{self.wingArea}$

IF aircraft.aspectRatio > 12 THEN

...wait a moment?

- These methods **are not inferring** new facts based on existing facts!
- They are **triggering some computation** to evaluate slot values!
- They are in fact **transparent to the inference mechanism**, which only bothers about matching and firing rules*
- *Yet...*



Inference mechanism in FBSs

The object-oriented paradigm applied to knowledge based systems.

Let's revisit previous example:

Aircraft	
wingSpan	float
wingArea	float
engineCount	1
range	float

IF engineCount = 2 AND
range <= 3000km
THEN category = "Regional Jet"

IF engineCount >=2 AND
range > 8000km
THEN category = "Long range Jet"

Aircraft	
wingSpan	float
wingArea	float
engineCount	1
range	float
category	IF self.engineCount = 2 AND self.range <= 3000km THEN "Regional Jet" ELSE ...

- Is this a production rule embedded in a method?
- Is this method inferring a new fact based on existing facts?*

Methods in FBSs

The object-oriented paradigm applied to knowledge based systems.

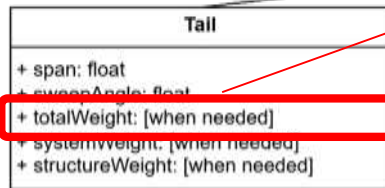
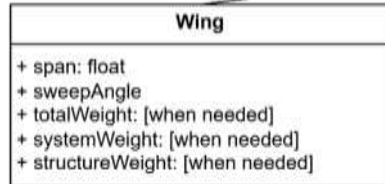
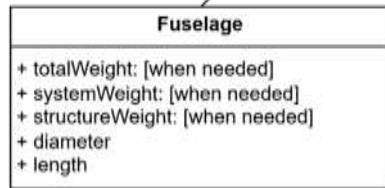
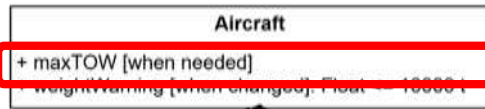
Who is executing these methods? and **when**?

When is indeed the right questions, because two types of methods are available in FBSs:

1. **when-needed*** methods are executed when the value of a given slot is requested and not directly available
→ **lazy evaluation (demand driven)**
2. **when-changed**** methods (**demons**) act as background reactive processes that execute when the value of a monitored slot changes
→ **eager evaluation (event-driven)**

Methods in FBSs [when-needed]

→ lazy evaluation

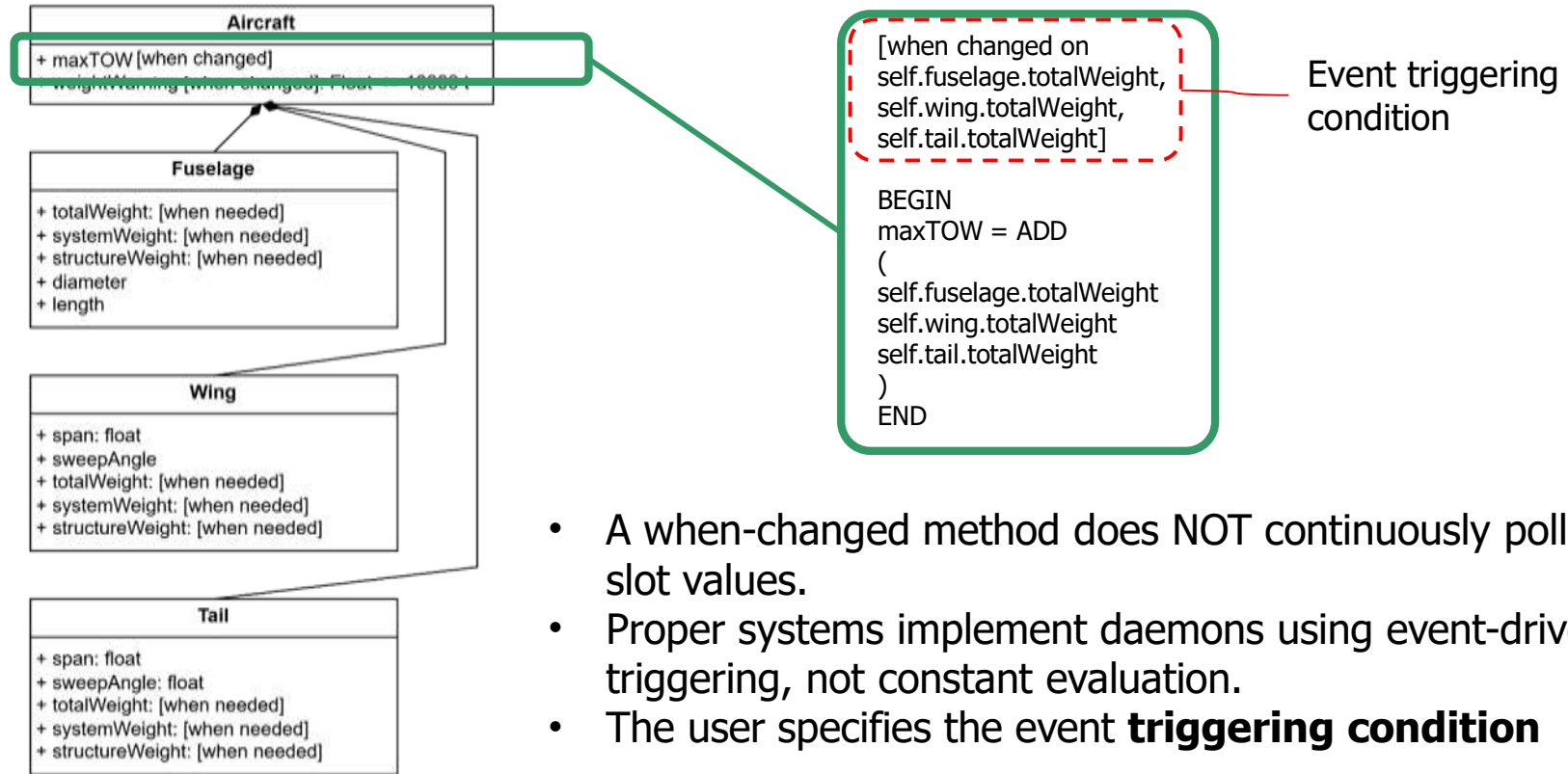


Reference chain

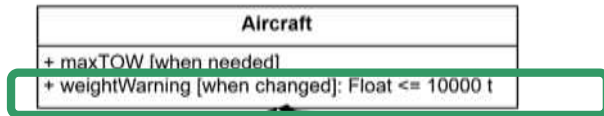
```
[when needed]
BEGIN
maxTOW = ADD
(
self.fuselage.totalWeight
self.wing.totalWeight
self.tail.totalWeight
)
END
```

```
[when needed]
BEGIN
totalWeight = ADD
(
self.systemWeight
self.structureWeight
)
END
```

Methods in FBSs [when-changed] → eager evaluation



Methods in FBSs [when-changed] → eager evaluation



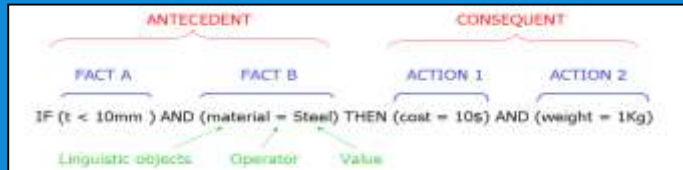
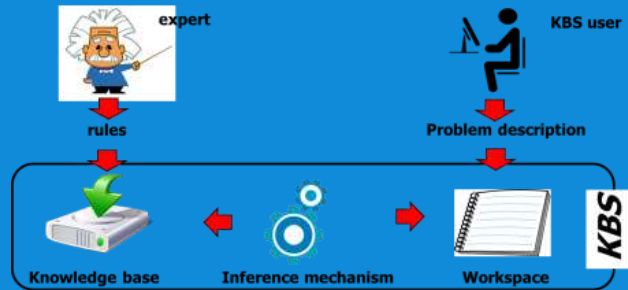
[when changed on self.maxMOW]
IF maxTOW > 10000 t
THEN
PRINT "Aircraft MTOW too high!"

Again, do I see a production rule?

- Although methods* may contain IF-THEN logic, they **are not** part of the global inference mechanism.
- They are local, deterministic, event-driven procedures attached to specific slots
- *Yet, the crisp separation between knowledge base and inference mechanism is getting blurry...*

Recap on KBSs:

Rules Based VS Frame Based Systems



Aircraft	
MTOW	Kg
No of passengers	
Wing span	m
Total length	m
Wing area	m ²
Range	Km

Fokker100: Aircraft	
MTOW	43000 Kg
No of passengers	107
Wing span	28 m
Total length	35.5 m
Wing area	93.5 m ²
Range	2500 Km

AirbusA380: Aircraft	
MTOW	560000 Kg
No of passengers	853
Wing span	79.8 m
Total length	73 m
Wing area	845 m ²
Range	15200 Km

Frame Based Systems (FBS) v.s. Rule Based Systems (RBS)

- RBSs organize knowledge mainly as flat collections of IF-THEN production rules
- FBSs allow the user to structure domain knowledge using networks of frames and relations (inheritance, composition), next to production rules
- FBSs provide greater modeling flexibility, but also greater modeling complexity
- Methods (procedural attachments) enable deterministic data processing in addition to symbolic rule reasoning
- ...

Frame Based Systems (FBS) v.s. Rule Based Systems (RBS)

- ...[continued]
- IF-THEN logic may appear both in production rules and inside methods
- The strict separation between knowledge base and inference mechanism becomes less sharp in FBSs



Limits of KBS to support engineering design

Since the beginning of the 70-ies, KBSs have started entering the software market. **Help-on-line, planning/scheduling systems and diagnostic tools** represent just some of **daily commodity KBSs**.



Medical diagnostics



Mineral analysis



Crops treatments



Linguistics



Planning and Scheduling



Financial Decision Making



Process Monitoring and Control

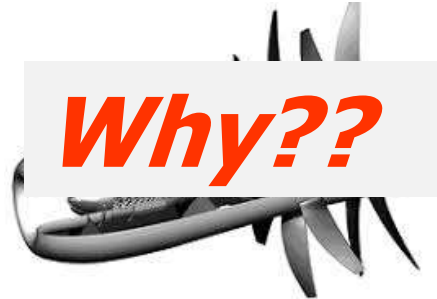


Knowledge
publishing



Limits of KBS to support engineering design

...however, to date, KBSs had **very limited impact in the field of Engineering Design**, i.e. automotive, aerospace and other fields concerned with the development of **complex hardware products**.



Limits of KBS to support engineering design

...because...

- KBSs have **limited capability** to represent and manipulate **geometry and shape information**
- KBSs provide **limited support for numerical analysis and simulation**

...whereas most engineering design work...

- requires input and produces output that poses **geometry creation and manipulation challenges**
- involves with **complex product (re)configuration**
- **must provide consistent data** to multiple discipline analyses
- **relies on** multidisciplinary analysis results **to drive design decisions**

Limits of KBS to support engineering design

KBSs reason well on symbols, but engineering design requires computation on geometry and physics.

At date, we have the following situation:

- Geometry computation → Computer Aided Design (**CAD**) tools
- Physics computation → Computer Aided Analysis (**CAA**) tools (FE, CFD, etc.)

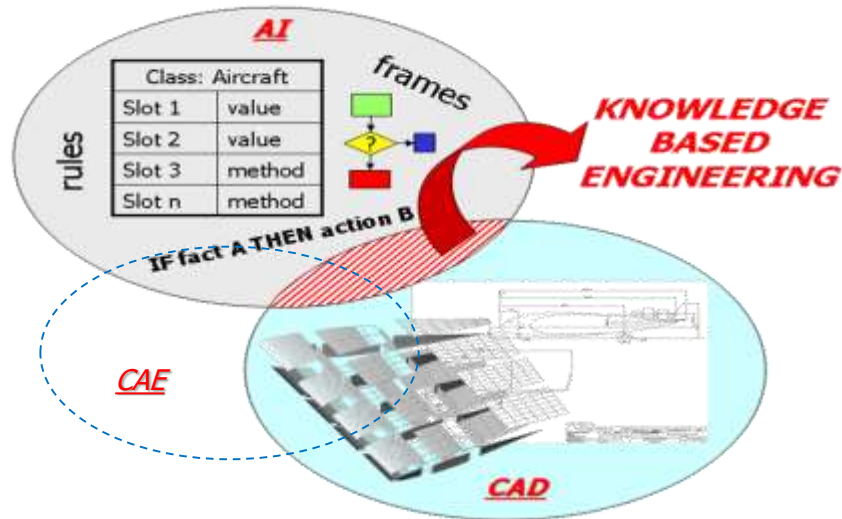
Do specialized systems exist that **combine the geometric and analysis power of CAD/CAA*** with the **knowledge capture and automation strengths of KBSs**?

Yes, these are known as **Knowledge Based Engineering (KBE) systems**

The origins of KBE

KBE systems are **an evolution of KB systems towards the specific needs of the engineering design domain.**

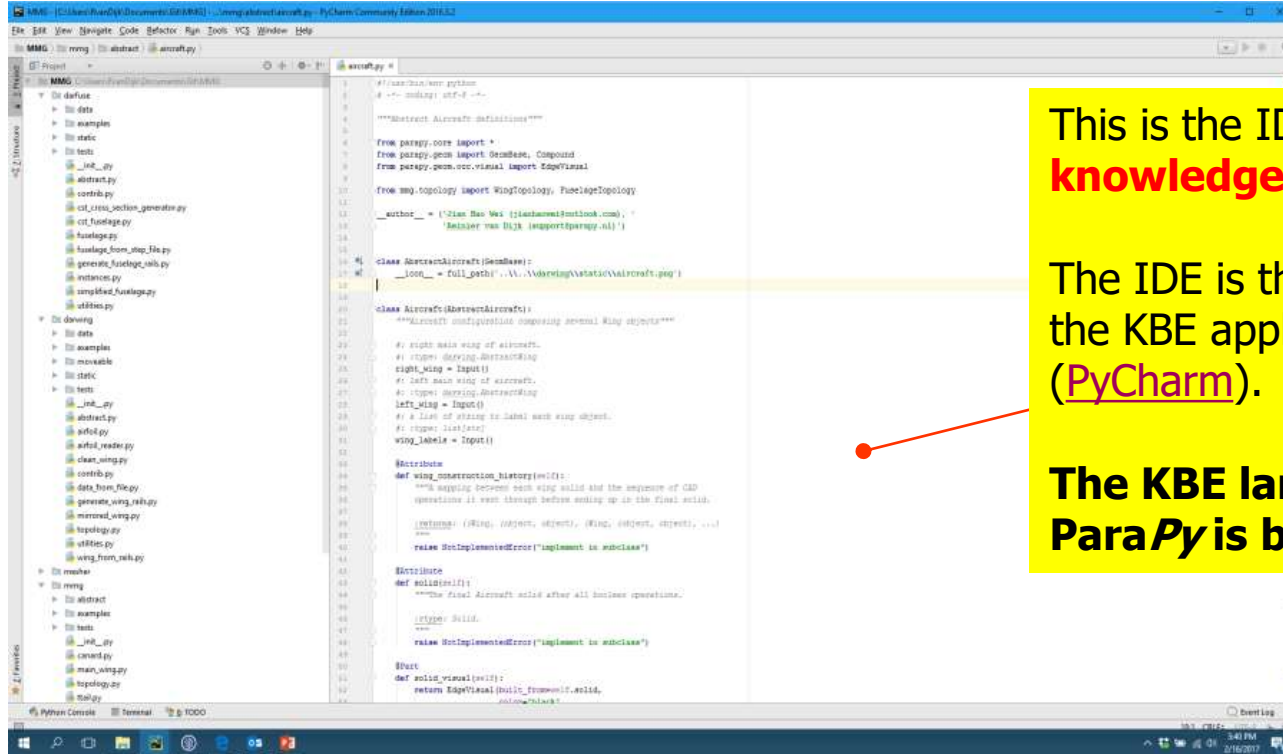
Their name derives from the fusion of **Knowledge Based Systems + Engineering → Knowledge Based Engineering systems***



KBE Definition #2:

A methodology that integrates knowledge based AI-derived techniques with CAD capabilities to enable automated engineering design and analysis

How does “doing KBE” look like?



```

1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 """Abstract Aircraft Definitions"""
5
6 from parapy.core import *
7 from parapy.geom import Database, Compound
8 from parapy.geom.vizual import EdgeVizual
9
10 from wing.topology import WingTopology, FuelageTopology
11
12 __author__ = ('Zian Bao Wei (zianbawei@outlook.com)',
13             'Reinier van Bijk (reiner@parapy.nl)')
14
15 class AbstractAircraft(BaseClass):
16     __doc__ = full_path('..\\..\\Addressing\\Vatatic\\Aircraft.ppt')
17
18
19 class Aircraft(AbstractAircraft):
20     """Aircraft configuration composing several Wing objects"""
21
22     # right main wing of aircraft
23     # type: deriving-abstract-wing
24     right_wing = Input()
25     # left main wing of aircraft
26     # type: deriving-abstract-wing
27     left_wing = Input()
28     # a list of string to label each wing object.
29     # type: list(str)
30     wing_label = Input()
31
32 @abstractmethod
33 def wing_construction_history(self):
34     """A mapping between each wing solid and the sequence of CAD
35     operations it went through before ending up in the final solid.
36
37     @return: ((Wing, object), (Wing, object, object), ...)
38     """
39     raise NotImplementedError("implement in subclass")
40
41 @abstractmethod
42 def solid(self):
43     """The final Aircraft solid after all business operations.
44
45     @type: Solid
46     """
47     raise NotImplementedError("implement in subclass")
48
49 @abstractmethod
50 def solid_vizual(self):
51     return EdgeVizual(solid_from(self.solid,
52                             color='black')

```

This is the IDE, **where the knowledge is authored**

The IDE is the tool to write the KBE application code (PyCharm).

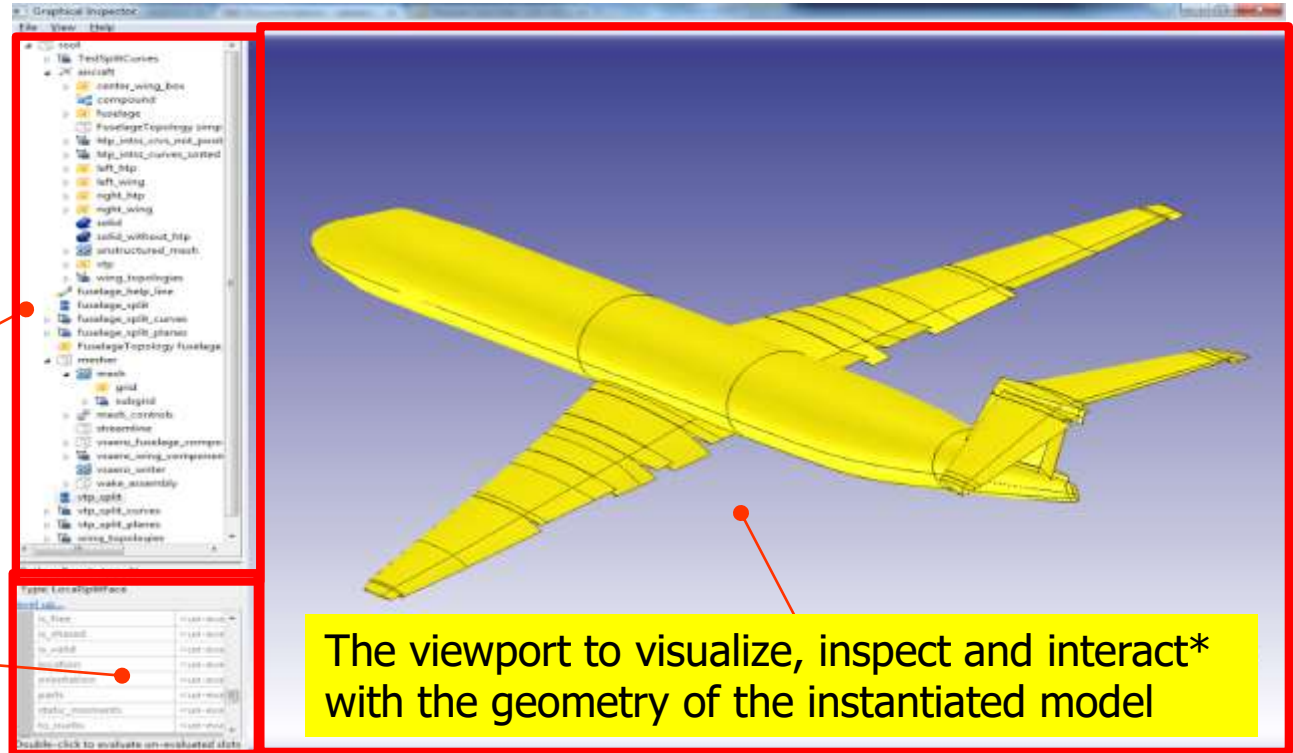
The KBE language used by ParaPy is built on Python

How does “doing KBE” look like?

The KBE runtime UI, where **knowledge is executed and explored**

The pane where the product tree of the instantiated model is displayed

The property grid, where attribute values can be inspected and input values interactively modified



The viewport to visualize, inspect and interact* with the geometry of the instantiated model

What KBE inherits from RBS/FBSs

- Knowledge formalization for reuse
- Object-centered modeling (class/object frames)
- Rule logic
- Methods (procedural attachments)
- Lazy/demand-driven evaluation

What KBE adds that RBS/FBSs lacked

- Geometry manipulation
- CAD/CAE integration
- Multidisciplinary analysis coupling
- Parametric/Generative modeling
- Scalable dependency management
- Engineering-grade automation

RBS/FBS → KBE: From symbolic AI → to knowledge-driven engineering computation

Main reference material for this lecture

- **Chapter 3** *Knowledge Based Engineering. The AI roots and the OO paradigm*
- **Journal paper** (*G. La Rocca, Knowledge based engineering: Between AI and CAD. Review of a language-based technology to support engineering design, Advanced Engineering Informatics, 26 (2012) pp159–179*)

All available in the section *Reading Material on*

